

Candidate Task — Data Widget, Cycle 2

Data engineering & operational assessment — Sportingrisk

Builds on the Cycle 1 prototype submission

Context

Cycle 1 was a prototype assessment. You delivered a working widget suite reading from mock JSON files.

Cycle 2 moves you toward the real architecture: an analytical database fed by a realistic ingestion path, with a cache layer in front, replicated for failover, with backups.

The user-visible behaviour does not change. The same 7 widget types you covered in Cycle 1 must continue to work, with the same API contract. Only the data layer changes — from mock JSON to a real ClickHouse cluster with ingestion, caching, replication, and backup.

This cycle evaluates **data-engineering and operational judgement**: schema design for analytical workloads, batching and idempotency in ingestion, cache discipline, and replication / failover behaviour. We are looking for decisions you can defend, not features you can list.

What you start with

- Your Cycle 1 submission — the working prototype with mock JSON data.
- The same 7 widget types you covered in Cycle 1 (no new widgets required).
- The same API contract — handlers should return the same response envelopes. Only the data source changes.

Part A — ClickHouse, Ingestion, Cache

Deliverables

1. **ClickHouse in docker-compose** alongside the API.
2. **Schema design** for the 7 widget types — one table per widget or normalised, your call, but you must justify the choice.
3. **Seed script** that converts your existing mock JSON into ClickHouse inserts. Cycle 1 data drives Cycle 2 widgets — no behavioural regression.

4. **DataProvider swap** — same interface, ClickHouse-backed. Mock provider remains available behind a config flag for tests.

5. **Live ingestion path** — at least one working flow. Choice of transport is yours: RabbitMQ consumer, HTTP `/ingest` endpoint, async insert with app-side buffer, or a hybrid. Transport is not what we score; the shape and handling of the payload is.

Payload shape: the inbound message structure should be equivalent to the API response structure for that resource. If a goal event mutates the live-stats payload that the widget reads, the inbound message should look like the relevant slice of that same payload — not a separate event vocabulary you then translate. This keeps the ingestion path, the storage layer, and the read API on a single shared data model.

Demonstrate end-to-end: an event arrives via your chosen transport → storage updates → cache invalidates → the widget reflects it within a second.

6. **Cache layer** in front of ClickHouse (Redis or in-process). The per-endpoint TTLs you declared in Cycle 1 must be honoured at the cache layer, not just in `Cache-Control` headers.

7. **Cache invalidation on ingest** — when a goal event lands, the relevant live-stats cache key is busted (or stale-while-revalidate equivalent).

Decisions you must make and justify

Each of the following must be addressed in the README or a design doc. We score on the reasoning, not the answer.

Decision	What we want to see
Table engine per widget	<code>MergeTree</code> , <code>ReplacingMergeTree</code> , <code>AggregatingMergeTree</code> , materialised views — different data shapes need different engines. One engine for everything is a fail.
Partition + ORDER BY	Partition by <code>toYYYYMM(kickoff)</code> ? By competition? <code>ORDER BY (match_id, ts)</code> for events vs <code>(competition_id, matchday)</code> for fixtures? This is where query latency lives.
Ingestion mechanism	Transport choice (RabbitMQ vs HTTP vs async insert) is up to you and is <i>not</i> what we score. We do score: backpressure behaviour, what happens when the consumer falls behind, and how the inbound payload shape maps to the API response shape with minimum translation.
Batching cadence	ClickHouse dies on per-event inserts. What's the buffer window — N rows or N ms? What happens on crash mid-buffer?
Idempotency	Same goal event arrives twice from the feed — what stops a double-count? Dedup key on <code>ReplacingMergeTree</code> ? <code>INSERT ... DEDUPLICATE</code> ? Application-level dedup table?

Materialised views	Live xG cumulative and rolling team stats are textbook MV cases. Do you reach for one? If not, why not?
Schema migrations	How does v2 of the schema ship without downtime? <code>ALTER TABLE</code> strategy or new-table-then-rename?
Backfill	If the ingestion service falls behind for an hour, how do you recover without double-writing or losing events?

Bonus signals

- TTL on raw events table (drop after season ends).
- `LowCardinality(String)` for team / competition IDs.
- Read-only DB user for the API; ingestion via a separate user.
- Load test or a basic benchmark in the README (rows/sec sustained, p95 query latency).
- Awareness of what does *not* belong in ClickHouse — e.g. client config is a small KV, not analytical.

Part B — Replication, Failover, Backup

Deliverables

1. **At least 2 ClickHouse replicas** in `docker-compose`, plus **ClickHouse Keeper** (preferred) or ZooKeeper for coordination.
2. **ReplicatedMergeTree family** in place of the plain engines from Part A — `ReplicatedReplacingMergeTree`, `ReplicatedAggregatingMergeTree`, etc.
3. **Distributed table layer** or app-side replica pool — the API does not need to know which replica answered.
4. **Failover demo** — a script or test that kills one replica during live ingestion and shows that reads continue and writes resume cleanly when the replica rejoins.
5. **Backup** — `clickhouse-backup` (or equivalent) on a schedule, with a restore script. Target an S3 bucket or a local volume.
6. **AWS deployment layout in the README** — replicas in separate AZs, Keeper ensemble (3 nodes, odd number), where the cache sits, security group rules.

Decisions you must make and justify

Decision	What we want to see
Replication factor	2 replicas (cheap, survives one node) vs 3 (survives one + maintenance window). For a single-region operator widget, defend your number.
Keeper topology	Standalone Keeper ensemble vs colocated with ClickHouse. Colocated is fine at our scale; standalone scales better. Even-numbered ensembles are a hard fail.
Replicas vs shards	We need replicas (redundancy), not shards (scale). A candidate who reaches for sharding at our data volume is over-engineering. Defend your choice either way.
<code>insert_quorum</code>	Default (=1) means an ack before replication — fast but a node loss equals lost rows. Quorum > 1 waits for a second replica before ack. For live match events, what's the right call?
<code>select_sequential_consistency</code>	After an ingest, a read on a different replica might be behind. Do you enable consistency for reads (slower) or accept the lag (faster, may show stale xG for ~1s)?
Read routing	<code>Distributed</code> round-robin, app-side replica pool, or external load balancer? Failover speed differs.
Failover detection	How fast does a dead replica get rotated out? Driver-level health checks, LB health checks, <code>Distributed</code> engine retry?
Adding a replica live	Production reality: scaling 2 → 3 replicas without downtime. Describe the procedure (<code>ATTACH</code> / <code>SYSTEM RESTORE REPLICA</code>).
Network partition	Keeper loses quorum — what happens to writes? Do you understand the read-only fallback?
Backup ≠ replication	Replication survives node loss, not <code>DROP TABLE</code> or schema corruption. Your backup must.

Bonus signals

- Replicas configured for separate AZs (not just two containers on one host).
- Distinct DB users for app, ingestion, admin.
- Monitoring: `system.replication_queue_lag`, replica delay alerts named.
- TLS between replicas and between API and ClickHouse.
- Choice of `clickhouse-keeper` over ZooKeeper, with reasoning (ZK is legacy in 2026).

What We Evaluate

Cycle 2 is about *judgement under realistic constraints*. Scoring axes:

1. **Schema fit** — right engine for each data shape, partition / ORDER BY chosen for query patterns.
2. **Ingestion robustness** — batching, idempotency, backpressure, recovery thinking.
3. **Cache discipline** — TTLs honoured at the cache, invalidation on ingest correct, no stampede risk.
4. **Failover behaviour** — actually demonstrated, not just claimed.
5. **Operational sense** — backups separate from replication, monitoring named, scaling path clear.
6. **README quality** — decisions and tradeoffs explained, not just a how-to-run section.

Out of Scope

- New widget types beyond the 7 from Cycle 1.
- UI / theming changes (unless a small frontend tweak is needed for live updates).
- Auth or multi-tenancy — the API is still public and read-only.
- Sharding ClickHouse — replicas only, single-shard cluster.
- Production-grade Keeper hardening, secrets management, full observability stack.
- Re-implementing the data feed itself — you simulate inbound events via a script.

Submission

- Repo or archive that runs end-to-end with `docker compose up --build`.
- README covering: how to run, how to ingest a test event, how to trigger failover, how to restore from backup.
- Design doc (in README or separate) addressing every decision listed in Parts A and B.
- Tests for the ingestion path and at least one failover scenario.

Questions: direct to your point of contact at Sportingrisk. Clarification questions on scope are welcome and expected.